

Circuitos secuenciales – Biestables

Hasta el momento, se han visto circuitos combinacionales, es decir, circuitos en los que la salida depende únicamente de los valores en las entradas; no son capaces de memorizar resultados producidos por sus combinaciones.

Un ejemplo es una compuerta *OR*, donde la salida S es una función que depende de las entradas A y B :



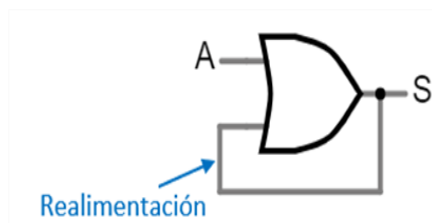
$$S = f(A, B)$$

A continuación, se verán circuitos secuenciales.

Circuito secuencial

Un circuito secuencial es aquel en el que su salida depende tanto del estado actual de la entrada como del estado anterior. Para esto, utilizan un lazo de retroalimentación, lo que permite que sean circuitos capaces de guardar información (*bits*).

Ver el siguiente ejemplo, donde la salida S es una función que depende tanto de la entrada A como del valor anterior de su salida S .



$$S_{n+1} = f(A, S_n)$$

S_n : estado actual de la salida

S_{n+1} : estado futuro de la salida (depende del actual)

¹ Circuito realizado en <https://www.falstad.com/circuit/>

² Circuito realizado en <https://www.falstad.com/circuit/>

Los circuitos secuenciales pueden ser sincrónicos o asincrónicos. Los circuitos secuenciales sincrónicos son aquellos en los que todos sus elementos están, como su nombre lo indica, sincronizados mediante una señal de reloj (*clock*). La señal de reloj (*clock*) es la que le permitirá cambiar de estado. La señal de *clock* es una señal cuadrada periódica (un tren de pulsos) que toma valor alto y bajo, como se muestra a continuación.



Señal de clock (reloj)

Los circuitos secuenciales asincrónicos, en cambio, no dependen de una señal de reloj (*clock*) para cambiar de estado. En cuanto reciba un cambio en sus entradas, cambiará su estado.

Biastable

Los biestables son los elementos más básicos dentro de los circuitos secuenciales. Estos son los elementos de memorias más chicos capaces de mantener o almacenar un *bit*. Como su nombre lo indica, pueden tomar dos estados estables: '0' y '1'.

Dentro de los biestables, se pueden diferenciar dos tipos: *latch* y *flip flop*.

Latch

Un *latch* es un elemento de memoria de un *bit* que al momento en el que cambian sus entradas podrá cambiar de estado (idealmente). En caso de tener una entrada de control (habilitación o *enable*), este podrá cambiar el estado siempre que esté habilitado para cambiarlo. Por lo tanto, se dice que estos últimos cambian de estados por nivel (es decir, cuando la entrada de control o *latch enable* es 1 o nivel alto (*High*)). Estos se suelen utilizar en los circuitos secuenciales asincrónicos.

Flip-Flop

Un *flip flop* es un elemento de memoria *bit* que cambia de estados según la señal del reloj. Dependiendo de la configuración, estos cambian de estado por flanco ascendente o

³ Diagrama realizado en <https://wavedrom.com/>

descendente del *clock* (reloj), es decir, por la transición o borde. Esto es más conveniente respecto del *latch*, ya que al cambiar de estado solo en el flanco resuelve problemas de estados transitorios inestables.

A continuación, se puede ver marcado en la señal de *clock* el flanco ascendente y flanco descendente de un pulso de *clock* (esto valdrá para todos los pulsos de *clock*).



Estos se suelen utilizar en los circuitos secuenciales sincrónicos.

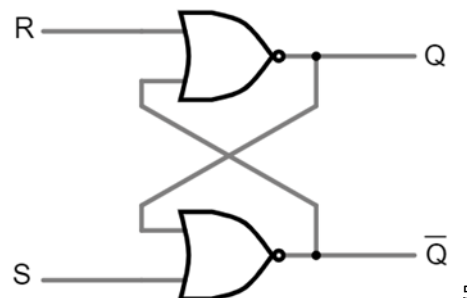
A continuación, se diferenciarán cuatro tipos de biestables según su funcionamiento: *SR*, *D*, *JK* y *T*

Latch *SR* (Set – Reset)

Aquí se desarrollará el *latch SR*. Este es un circuito que en su estado SET (“setea”) le asignará un 1 a la salida y en su estado RESET (“resetea”) le asignará un 0 a la salida, o en otro caso retendrá el valor de su salida.

▪ Diagrama lógico

Se realizará el circuito del *latch SR* con compuertas *NOR*. Sin embargo, también es posible plantearlo con compuertas *NAND*.



⁴ Diagrama realizado en <https://wavedrom.com/>

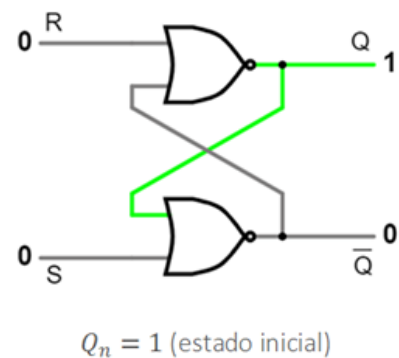
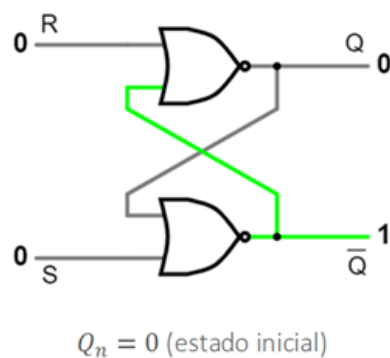
⁵ Circuito realizado en <https://www.falstad.com/circuit/>

▪ *Tabla de verdad*

Para armar la tabla que se corresponde al funcionamiento del anterior diagrama lógico, será necesario darle valores a R, S y Q_n (valor inicial) para obtener el valor de la salida Q_{n+1} (valor futuro o siguiente).

- Si $S = 0$ y $R = 0$

En los siguientes circuitos, el valor de Q es el resultado (estado futuro) según el estado inicial indicado debajo de estos.

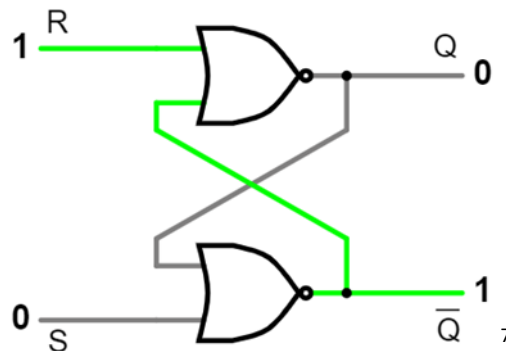


6

Por lo que, en el caso de que tanto S y R sean ambos 0, el valor del estado futuro Q_{n+1} será igual a Q_n (estado inicial).

- Si $S = 0$ y $R = 1$

Si se recuerda el funcionamiento de la compuerta NOR , si alguna de las entradas está en 1, entonces la salida de esta es 0; y si ambas entradas son 0, la salida es 1. Por lo que sea el estado inicial $Q_n = 0$ o $Q_n = 1$, se obtiene lo siguiente.

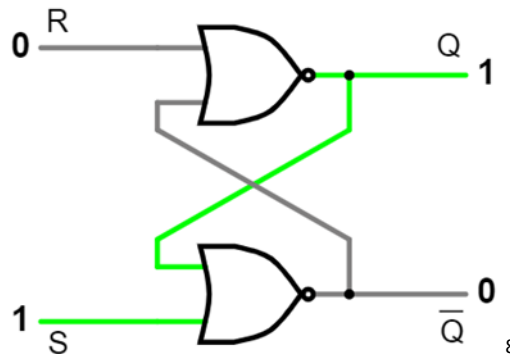


⁶ Circuitos realizados en <https://www.falstad.com/circuit/>

Entonces, cuando la entrada del *Reset* sea 1 ($R = 1$) y la del *Set* sea 0, en la salida se obtendrá un 0 ($Q_{n+1} = 0$), sin importar el valor inicial de Q_n .

- Si $S = 1$ y $R = 0$

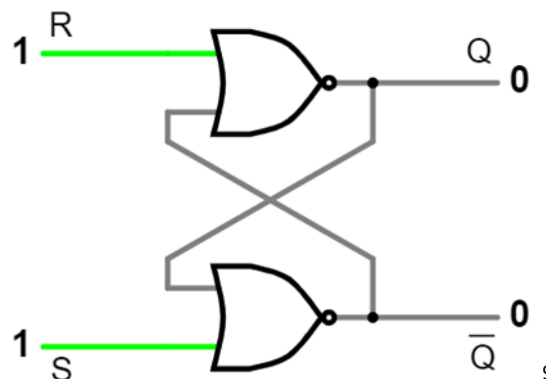
Siguiendo el mismo razonamiento que antes, tanto si el estado inicial es $Q_n = 0$ o $Q_n = 1$, se obtiene lo siguiente.



Entonces, cuando la entrada del *Set* sea 1 ($S = 1$) y la del *Reset* sea 0, en la salida se obtendrá un 1 ($Q_{n+1} = 1$), sin importar el valor inicial de Q_n .

- Si $S = 1$ y $R = 1$

Puesto que ambas compuertas *NOR* tienen una entrada en 1, la salida de ambas será siempre 0, obteniéndose el siguiente circuito, donde Q y $\bar{Q}$ tienen el mismo valor a la salida.



⁷ Circuito realizado en <https://www.falstad.com/circuit/>

⁸ Circuito realizado en <https://www.falstad.com/circuit/>

⁹ Circuito realizado en <https://www.falstad.com/circuit/>

Como Q y $\bar{Q}$ deben ser complementarias, es decir, si una es 0 la otra debe ser 1, y viceversa, entonces se dice que para este tipo de biestable SR el estado donde $S = 1$ y $R = 1$ es un estado prohibido. Este estado prohibido lo marcaremos con una X en la tabla.

A partir de todo este análisis es posible armar la tabla de verdad del biestable SR . A continuación, se verán dos tablas, una más simplificada que la otra.

Tabla de verdad SR

Tabla ampliada

S	R	Q_n	Q_{n+1}	
0	0	0	0	Q_n
0	0	1	1	
0	1	0	0	0
0	1	1	0	
1	0	0	1	1
1	0	1	1	
1	1	0	X	X
1	1	1	X	
S	R	Q_n	Q_{n+1}	

No
permitido

Tabla reducida

S	R	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	X

▪ Expresión lógica

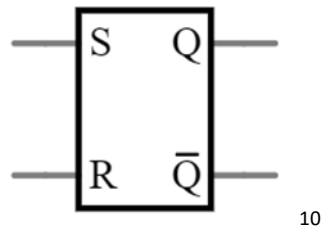
A partir de la tabla de verdad se puede simplificar y obtener la ecuación característica correspondiente al biestable SR .

$R \ Q_n$		00	01	11	10
S	0		1		
	1	1	1	X	X

$$Q_{n+1} = S + \bar{R} \cdot Q_n$$

- *Símbolo lógico*

El símbolo lógico del *latch SR* es el siguiente:

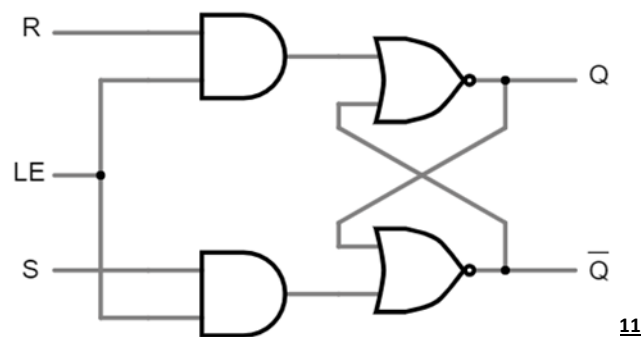


10

Latch SR (Set – Reset) con habilitación

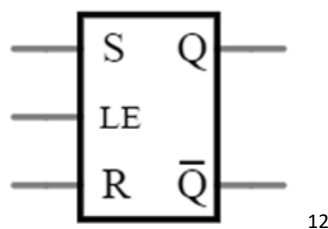
Al circuito anterior se le agrega la habilitación o *Latch Enable (LE)*, es decir, una entrada de control que indica si puede o no cambiar de estado.

- *Diagrama lógico*



11

- *Símbolo lógico*



12

¹⁰ Circuito realizado en <https://www.falstad.com/circuit/>

¹¹ Circuito realizado en <https://www.falstad.com/circuit/>

¹² Circuito realizado en <https://www.falstad.com/circuit/>

- *Tabla de verdad*

Cuando el *Latch Enable* esté en uno ($LE = 1$), permitirá que se produzca el cambio según el valor de las entradas. En cambio, cuando sea cero ($LE = 0$), no importa el valor de las entradas R y S , la salida mantendrá su estado.

Tabla ampliada

LE	S	R	Q_n	Q_{n+1}
0	X	X	0	0
0	X	X	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	X
1	1	1	1	X

Tabla reducida

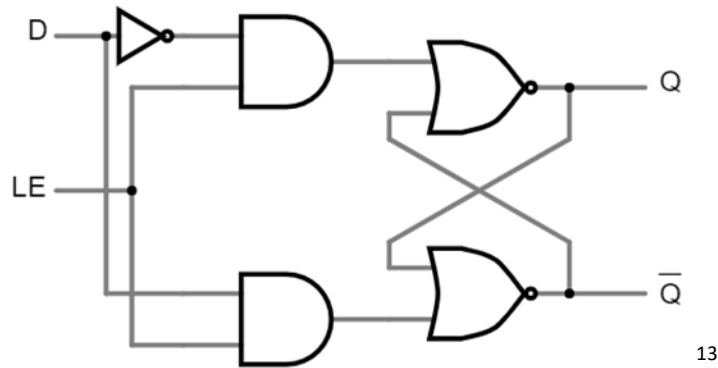
LE	S	R	Q_{n+1}
0	X	X	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	X

Latch D (Data) con habilitación

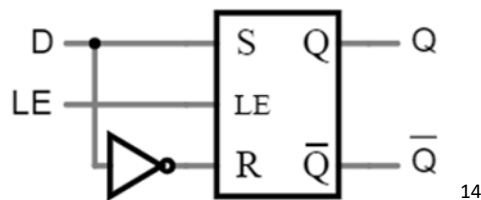
El *latch D* recibe su nombre de su entrada de datos. Se suelen utilizar para almacenar datos (*bits* de información), debido a su comportamiento.

- *Diagrama lógico*

El diagrama lógico correspondiente al *latch D* con habilitación es el siguiente:



Lo que es lo mismo plantearlo a partir del *latch* SR con habilitación.



■ Tabla de verdad

Para armar la tabla de verdad, se deberá tener en cuenta la tabla del *latch* SR.

En el caso de que $D = 0$, entonces $S = 0$ y $R = 1$, por lo que la salida será 0, sin importar el estado inicial de Q_n .

En el caso de que $D = 1$, entonces $S = 1$ y $R = 0$, por lo que la salida será 1, sin importar el estado inicial de Q_n .

La tabla del biestable D será la siguiente

Tabla ampliada

D	Q_n	Q_{n+1}	
0	0	0	0
0	1	0	
1	0	1	1
1	1	1	

Tabla reducida

D	Q_{n+1}
0	0
1	1

¹³ Circuito realizado en <https://www.falstad.com/circuit/>

¹⁴ Circuito realizado en <https://www.falstad.com/circuit/>

Teniendo en cuenta la habilitación, entonces se tiene el siguiente resultado.

Tabla ampliada

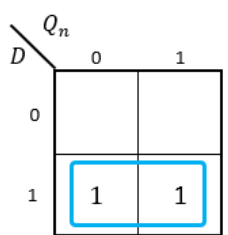
LE	D	Q_n	Q_{n+1}	
0	X	0	0	Q_n
0	X	1	1	
1	0	0	0	0
1	0	1	0	
1	1	0	1	1
1	1	1	1	

Tabla reducida

LE	D	Q_{n+1}
0	X	Q_n
1	0	0
1	1	1

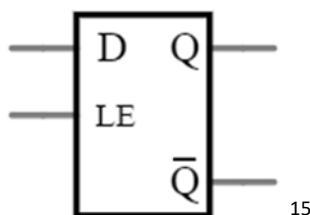
▪ Expresión lógica

A partir de la tabla de verdad del biestable se puede simplificar y obtener la ecuación característica correspondiente al biestable D.



$$Q_{n+1} = D$$

▪ Símbolo lógico



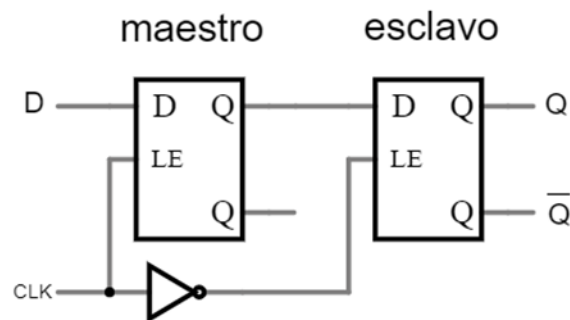
15

¹⁵ Circuito realizado en <https://www.falstad.com/circuit/>

Flip Flop **D** (Data) maestro – esclavo

Este tipo de *flip flop* se compone de dos *latch* D. Uno es el maestro (el cual recibe el dato de entrada), y luego le pasa este al siguiente *latch* (el esclavo), que a su salida mostrará el dato. Estos eliminan condiciones transitorias inestables de los circuitos vistos hasta el momento en este apunte.

▪ Diagrama lógico

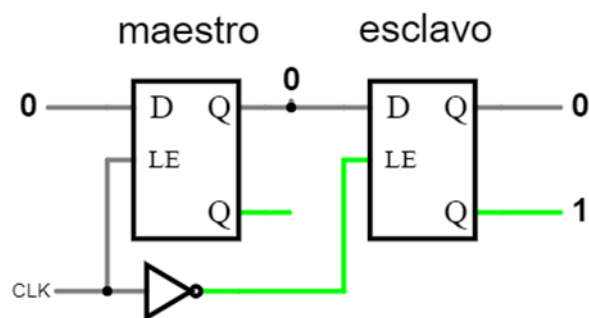


16

▪ Funcionamiento

Para entender el funcionamiento del circuito, será conveniente ponerle valores a D y ver qué pasa con los cambios del *clock* (reloj).

Supongamos que en un primer instante se tiene el siguiente circuito:

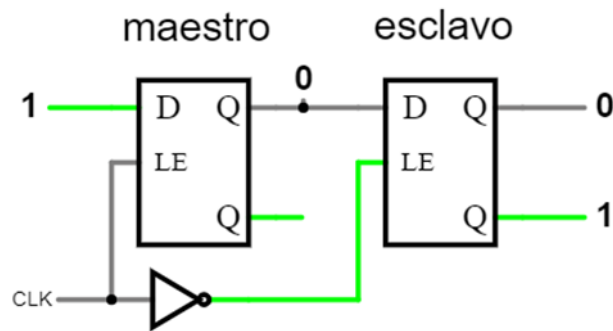


17

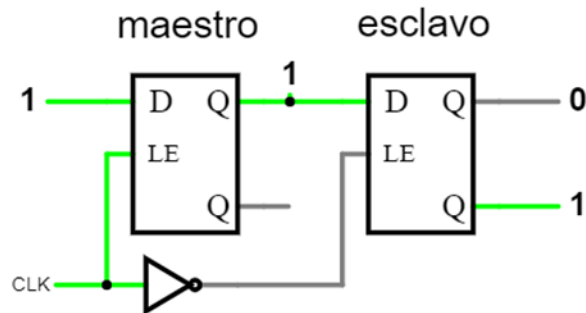
¹⁶ Circuito realizado en <https://www.falstad.com/circuit/>

¹⁷ Circuito realizado en <https://www.falstad.com/circuit/>

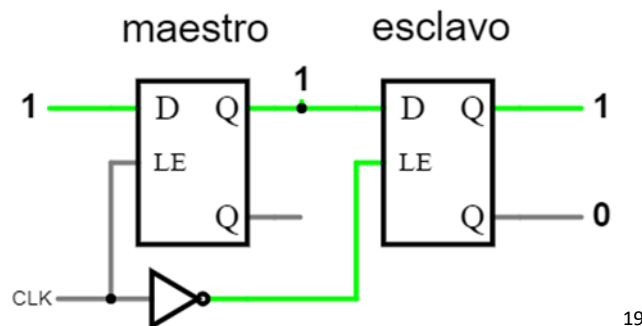
En este otro instante, la entrada del *latch* D maestro cambia a 1.



Recién cuando el *clock* esté en 1, habilitará a que cambie su estado obteniendo un 1 a la salida del *latch* D maestro y, por lo tanto, un 1 a la entrada del *latch* D esclavo.



Finalmente, cuando el *clock* esté en 0, al pasar por la *NOT* a la entrada del *latch enable* se recibirá un 1. Con lo cual habilitará a que este pueda cambiar su estado, obteniendo así un 1 a la salida del *latch* D esclavo.

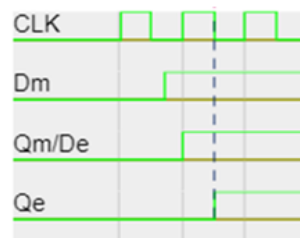


¹⁸ Circuito realizado en <https://www.falstad.com/circuit/>

¹⁹ Circuito realizado en <https://www.falstad.com/circuit/>

- *Diagrama temporal*

Ver el diagrama temporal correspondiente al análisis del funcionamiento anterior:



CLK: clock o reloj

Dm: entrada D del latch maestro

*Qm/De: salida Q del latch maestro y
entrada D del latch esclavo*

Qe: salida Q del latch esclavo

20

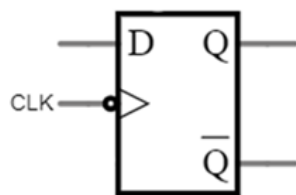
Observar que si bien el cambio en la entrada del circuito se produce en un instante, recién cuando llega a la transición descendente próxima se podrá apreciar el cambio a la salida del circuito. Esto se ve cuando se produce el cambio en *Dm* (que es la entrada del circuito) y cuando se produce el cambio en *Qe* (que es la salida del circuito).

Cabe destacar que este tipo de conexión no es la única forma de lograr este funcionamiento, ni la óptima.

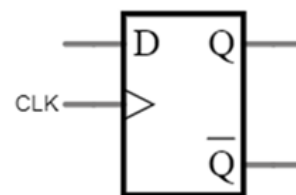
Flip Flop D (Data) por flanco

- *Símbolo lógico*

Dependiendo de su configuración, se podrá tener dos tipos de *Flip Flop D* por flanco descendente o por flanco ascendente del *clock*.



Flanco Descendente ↓



Flanco Ascendente ↑

21

²⁰ Circuito realizado en <https://www.falstad.com/circuit/>

²¹ Circuito realizado en <https://www.falstad.com/circuit/>

Nótese la diferencia con el *latch* antes visto. Para indicar la entrada dinámica (a la que se conecta el *clock*) del *flip flop* tendrá un triángulo y, en caso de que corresponda, un círculo.

Esta distinción del símbolo lógico para el caso de flanco ascendente o descendente será válida también para todos los tipos de *flip flop* que se verán.

■ *Tabla de verdad*

Aquí se muestra la tabla para el caso del flanco descendente, pero de igual manera se cumple para el flanco ascendente.

Tabla ampliada

<i>CLK</i>	<i>D</i>	Q_n	Q_{n+1}	
0	<i>X</i>	0	0	Q_n
0	<i>X</i>	1	1	
1	<i>X</i>	0	0	Q_n
1	<i>X</i>	1	1	
↓	0	0	0	0
↓	0	1	0	
↓	1	0	1	1
↓	1	1	1	

Tabla reducida

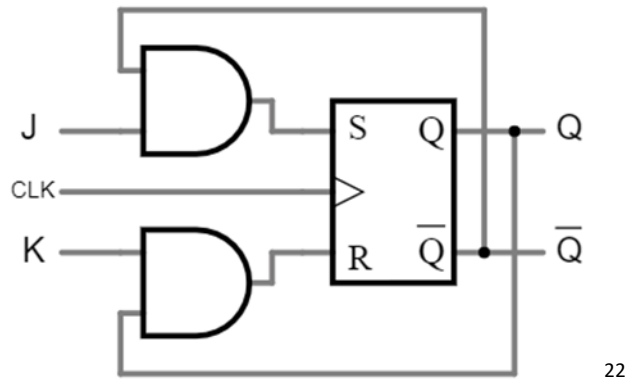
<i>CLK</i>	<i>D</i>	Q_{n+1}
0	<i>X</i>	Q_n
1	<i>X</i>	Q_n
↓	0	0
↓	1	1

Flip Flop JK (Jack Kilby) por flanco

Este tipo de *flip flop* recibe su nombre por Jack Kilby, ingeniero e inventor de los circuitos integrados. El *flip flop JK* es análogo al *flip flop SR* pero, como se verá a continuación, resuelve el cambio de estado cuando ambas entradas son 1.

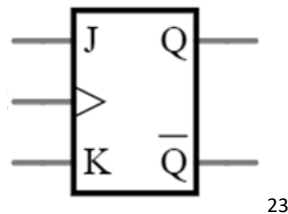
- *Diagrama lógico*

A partir del *flip flop SR* es posible armar el *flip flop JK*:



- *Símbolo lógico*

En este caso se muestra el símbolo del *JK* por flanco ascendente $\uparrow$. Si fuera descendente, bastaría indicarlo con el círculo de negación, tal como se mostró en el *flip flop D*.



- *Tabla de verdad*

A continuación, se muestra la tabla del biestable *JK*, con el objetivo de ver su funcionamiento. Tal como se hizo anteriormente, se puede agregar la columna del CLK.

²² Circuito realizado en <https://www.falstad.com/circuit/>

²³ Circuito realizado en <https://www.falstad.com/circuit/>

Tabla ampliada

J	K	Q_n	Q_{n+1}	
0	0	0	0	Q_n
0	0	1	1	
0	1	0	0	0
0	1	1	0	
1	0	0	1	1
1	0	1	1	
1	1	0	1	$\overline{Q_n}$
1	1	1	0	

Tabla reducida

J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	$\overline{Q_n}$

Ver que es similar a la tabla del biestable SR , pero resuelve el estado en el que las entradas son ambas 1. Se podría considerar a este una mejora del SR , dado que elimina el estado prohibido, donde S es equivalente a J y R es equivalente a K .

▪ *Expresión lógica*

A partir de la tabla de verdad se puede simplificar y obtener la ecuación característica correspondiente al biestable JK .

		$K Q_n$			
J		00	01	11	10
	0		1		
	1	1	1		1

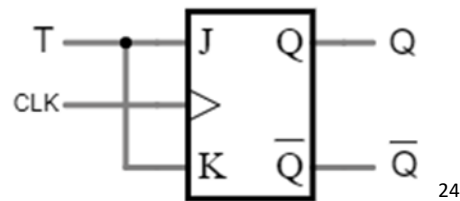
$$Q_{n+1} = J \cdot \overline{Q_n} + \overline{K} \cdot Q_n$$

Flip Flop T (toggle) por flanco

El *flip flop* T recibe su nombre por su significado en inglés, *toggle*, es decir, conmutar o alternar. Este mantiene o invierte (alterna) su estado, como se podrá ver a continuación.

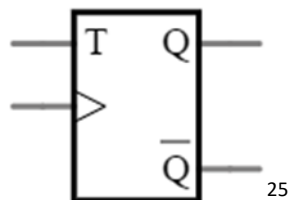
■ Diagrama lógico

Se partirá de un *flip flop* JK, para realizar el diagrama lógico del *flip flop* T.



■ Símbolo lógico

El símbolo del *flip flop* T por flanco ascendente $\uparrow$ es el siguiente:



■ Tabla de verdad

En esta oportunidad se muestra la tabla del biestable T , con el objetivo de ver su funcionamiento. Tal como se hizo anteriormente, se puede agregar la columna del CLK.

²⁴ Circuito realizado en <https://www.falstad.com/circuit/>

²⁵ Circuito realizado en <https://www.falstad.com/circuit/>

Tabla ampliada

T	Q_n	Q_{n+1}	
0	0	0	Q_n
0	1	1	
1	0	1	$\overline{Q_n}$
1	1	0	

Tabla reducida

T	Q_{n+1}
0	Q_n
1	$\overline{Q_n}$

Cuando $T = 0$ mantiene su estado, mientras que cuando $T = 1$ alterna, invierte su estado.

▪ *Expresión lógica*

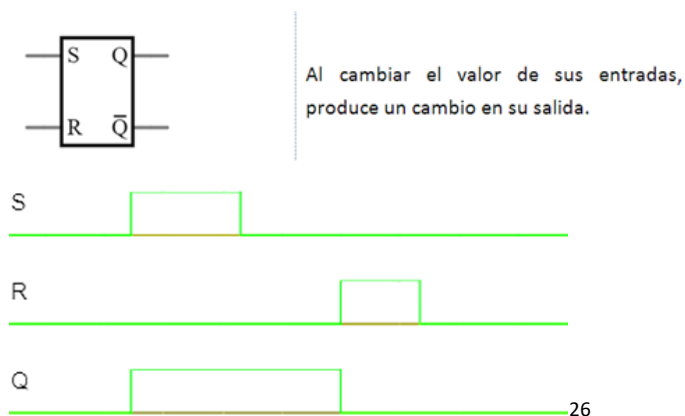
A partir de la tabla de verdad se puede obtener la ecuación característica correspondiente al biestable T.

$$Q_{n+1} = \bar{T} \cdot Q_n + T \cdot \overline{Q_n}$$

Diagramas temporales

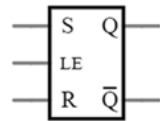
A continuación, se observarán los diagramas temporales de algunos de los diferentes biestables que se han visto. Es decir, el nivel de tensión en función del tiempo.

▪ *Latch SR*

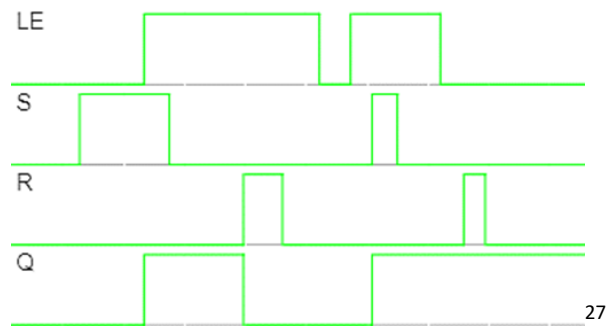


²⁶ Circuitos realizados en <https://www.falstad.com/circuit/>

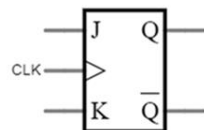
- *Latch SR con habilitación*



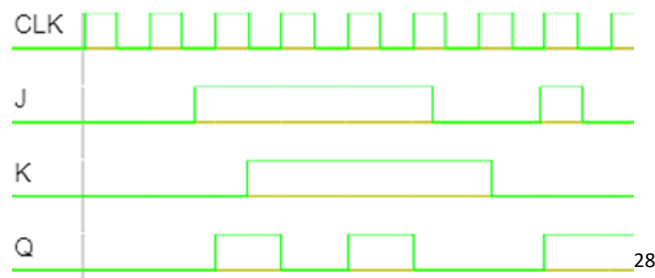
Solo se podrán producir los cambios cuando el *latch enable* (*LE*) se encuentre en nivel alto, es decir, en 1.



- *Flip Flop JK por flanco ascendente*



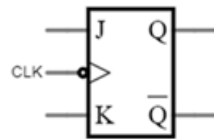
Podrá cambiar de estado según el valor de las entradas al momento que haya un flanco ascendente en la señal de *clock*.



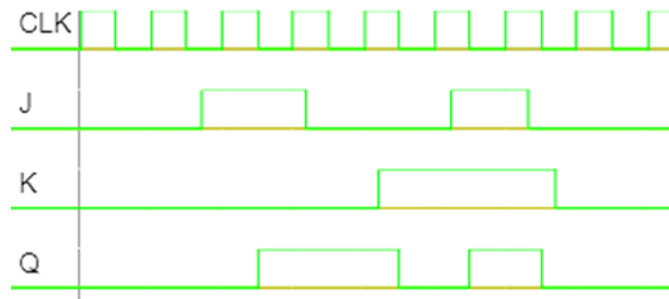
²⁷ Circuitos realizados en <https://www.falstad.com/circuit/>

²⁸ Circuitos realizados en <https://www.falstad.com/circuit/>

- *Flip Flop JK por flanco descendente*



Podrá cambiar de estado según el valor de las entradas al momento que haya un flanco descendente en la señal de *clock*.



Tablas de transiciones

Anteriormente, se han visto las tablas de funcionamiento de los biestables. A veces, será más cómodo trabajar con las tablas de transiciones, es decir, plantear la tabla según el valor de Q_n y Q_{n+1} (el estado actual y el estado futuro). A partir de estos, ver qué valor tendrían las entradas D, T, J y K.

- *Tabla de Transiciones del biestable T*

Para armar la tabla de transiciones, se deberá tener en cuenta la tabla ampliada del biestable. Se deberá mirar el estado actual y el futuro y ver en ese caso qué valor tiene T.

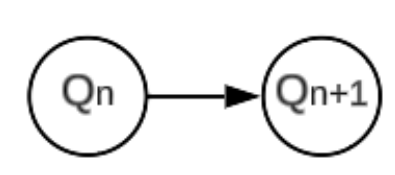


Tabla ampliada

T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

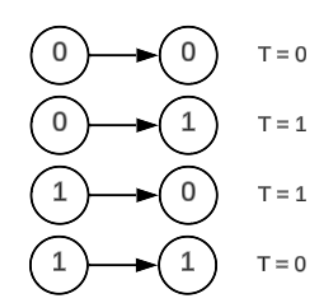


Tabla de transiciones

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

■ *Tabla de Transiciones del biestable D*

Teniendo en cuenta que la ecuación característica del biestable D es $Q_{n+1} = D$, se obtiene:

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

■ *Tabla de Transiciones del biestable JK*

Para armar la tabla de transiciones, se deberá tener en cuenta la tabla ampliada del biestable. Se deberá mirar el estado actual y el futuro y ver, en ese caso, qué valor tiene J y K .

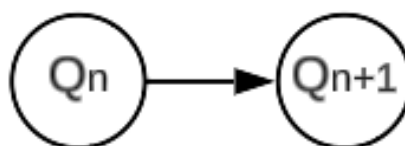


Tabla ampliada

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

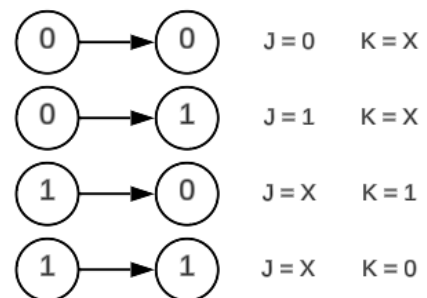


Tabla de transiciones

Q_n	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Nótese, por ejemplo, cuando el estado actual es cero ($Q_n = 0$) y el estado futuro es cero ($Q_{n+1} = 0$), en la tabla ampliada del biestable hay dos filas que cumplen con esta condición. En ambas $J = 0$, pero en un caso $K = 0$ y en el otro $K = 1$, por lo que en la tabla se pondrá $K = X$ (podrá tomar ambos valores).

Ejemplo: construir un flip flop a partir de otro

A partir de un flip flop D se quiere implementar un flip flop JK.

Para realizar este ejemplo, primero, se deberá tener en cuenta la tabla del biestable JK.

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

A esta tabla se le agrega una columna de nombre D (que es el *flip flop* del que se partirá), la cual se completará mirando los valores de Q_n y Q_{n+1} . Para completarla, será de utilidad tener en cuenta la tabla de transiciones del *flip flop* D.

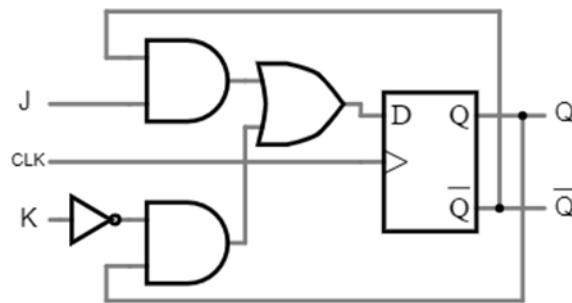
J	K	Q_n	Q_{n+1}	D
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

Simplificando el resultado de D de la tabla (siendo las variables J , K y Q_n), se obtiene:

$J \backslash K Q_n$	00	01	11	10
0		1		
1	1	1		1

$$D = J \cdot \overline{Q_n} + \overline{K} \cdot Q_n$$

Por lo que el diagrama lógico correspondiente a esta implementación será el siguiente.



29

²⁹ Circuito realizado en <https://www.falstad.com/circuit/>